MÓDULO DE PRACTICAS
24-25

INFORMATIZANDO UNA CADENA DE PIZZERÍAS

Adrián Gutiérrez García

PL6

UO300627

TABLA DE CONTENIDOS

Tabla de contenido

Introducción 3

Desarrollo de la apliacación 4

Lógica 4

Clase Pizzería 4

Clase Pedido 5

Clase Juego 6

Clase Tablero 7

Clase Casilla 7

Clase Carta 7

Clase Artículo 8

Clase Reservas 8

Clase Reserva 9

Clase ClavesEmpleados 9

Clase ClaveEmpleado 9

Interfaz 10

Ventana registro 10

Ventana recuperar datos de reserva 11

Ventana menú principal 12

Ventana nueva reserva 14

Ventana información artículos 15

Ventana tramitar pedido 16

Ventana juego 17

Ventana premios disponibles 18

Ventana premios obtenidos 19

Pruebas 20

INTRODUCCIÓN

El problema propuesto consiste en informatizar la gestión de comandas de una pizzería mediante la creación de una aplicación que estará disponible en la pantalla que habrá en cada una de las mesas del restaurante. Además, también habrá disponible un juego para amenizar la espera de la comanda solicitada. En mi caso he optado por no incluir la opción para subir nota de la internacionalización, y no he incluido ningún otro elemento que se deba tener en cuenta a la hora de corregir.

DESARROLLO DE LA APLICACIÓN

LÓGICA:

Clase Pizzería:

Esta es la clase servicio donde tenemos todos los métodos que conectan la interfaz con la lógica de la aplicación. Tendremos siempre una instancia de esta clase en la clase de la ventana principal. Tenemos como parámetros el pedido que corresponde con la reserva que se este llevando a cabo en ese momento, una instancia de la carta del restaurante, un objeto reservas para poder acceder a la lista de reservas del restaurante y un objeto clavesEmpleados para poder acceder a la lista de claves de los empleados del restaurante.

Como métodos más importantes tenemos:

- **Void añadirAPedido(int indice)** y **void eliminarDePedido(int indice)**: nos permite como bien dice el nombre añadir y eliminar artículos del pedido. Se pasa como parámetro el índice en la carta del artículo a añadir o eliminar. Estos métodos se desarrollarán en la clase pedido.
- **Void inicializar():** se utiliza para preparar una nueva reserva, actualizando la carta y la lista de reservas. Este método se acabará de desarrollar en la clase pedido.
- **Void eliminarReserva(String dni, String numReserva):** elimina la reserva actual dela lista de reservas. Se pasa como parámetros el dni y el número de reserva de la reserva actual. Este método se acabará de desarrollar en la clase reservas.
- **Void añadirArticulosFiltro(), void eliminarArticulosFiltro() y boolean estaEnFiltro():** son métodos que nos permiten filtrar los artículos según las categorías seleccionadas en el filtro del menú principal. Estos métodos se acabarán de desarrollar en la clase carta.
- **Void iniciarNuevaComanda():** este método nos permite resetear los datos para iniciar una nueva comanda. Este método se desarrollará en la clase pedido.
- **Void seleccionCasilla(int índice):** método que guarda las casillas que han sido seleccionadas por el usuario para intentar realizar pareja, tanto la primera como la segunda. Se pasa como parámetro el índice de la casilla seleccionada. Este método se desarrolla en la clase juego.
- **Boolean comprobarParejas(int índice):** método que comprueba si se ha hecho pareja. Se pasa como parámetro el índice de la casilla seleccionada. Este método se desarrolla en la clase juego.

- **Boolean comprobarFinJuego():** método que comprueba si ha terminado el juego. Este método se desarrolla en la clase pedido.
- **Void añadirPremio():** método que añade un premio en caso de que el cliente haya hecho una pareja. Este método se desarrolla en la clase juego.
- **Void limpiarListaArticulosFiltrados():** método que nos permite reiniciar el filtro vaciando la lista que contiene los elementos de las casillas seleccionadas en el filtro. Este método se desarrolla en la clase carta.
- **AñadirArticulosFiltroSoloIntolerancias(String actionCommand):** método que nos permite adaptar el filtro a la posibilidad de desarrollar un producto adaptado a intolerancias. Se pasa como parámetro el actionCommnand del artículo a añadir o eliminar que se corresponde con su categoría. Este método se desarrollará en la clase carta.

Clase Pedido:

En esta clase tenemos todos los métodos relacionados con la reserva actual y sus respectivos pedidos. Como atributos tenemos un objeto artículo que almacena la pizza del día, el importe total del pedido de tipo float, lista pedido que almacena todos los artículos que se han ido añadiendo al pedido, un objeto carta, un objeto reserva para almacenar el dni y el número de la reserva actual, un objeto juego, el número de comanda que se está procesando en ese momento y las observaciones que se incluyen al terminar una comanda.

Como métodos más importantes tenemos:

- **Void inicializarPedido():** método que inicializa todos los datos para empezar una nueva reserva: actualiza la carta, crea un nuevo objeto reserva sin ninguna reserva asociada hasta el registro, crea un nuevo juego, limpia la lista del pedido, pone el importe total a 0 y las observaciones como cadena vacía, pone el número de comanda a 0 y genera una nueva pizza del día.
- **Articulo generarPizzaDelDia():** método que selecciona una pizza aleatoria entre las que hay en la carta generando un número aleatorio y seleccionando en una lista, con solo las pizzas de la carta, la que está en la posición indicada por ese número aleatorio. Al precio de esa pizza se le aplica el 20% de descuento. Devuelve la pizza del día con el descuento aplicado.
- **Void añadirAPedido(int indice):** método que almacena en una variable el artículo correspondiente al índice que se pasa como parámetro. Después se recorre la lista del pedido, en caso de que ya este ese artículo, solo se sumará una unidad a ese artículo, y sino se añadirá ese articulo con una unidad a la lista del pedido. Finalmente se actualiza el importe total sumándole el precio del articulo añadido.

- **Void eliminarDePedido(int indice):** método que almacena en una variable el artículo correspondiente al índice que se pasa como parámetro. Después se recorre la lista del pedido, en caso de que ya este ese artículo, se restará una unidad, y en caso de que quede en 0 unidades se eliminará de la lista del pedido. Si no se encontró ese articulo en la lista no pasa nada. Finalmente se actualiza el importe total restándole el precio del artículo eliminado del pedido en caso de que se haya eliminado alguno.

- **Void iniciarNuevaComanda():** limpia la lista del pedido y pone el importe total a 0 para poder iniciar una nueva comanda.
- **Void añadirPizzaDelDia() y void eliminarPizzaDelDia():** mismo funcionamiento que el añadir y eliminar normal, pero en la variable inicial se almacena la pizza del día.
- **Void registrarComanda(String observaciones):** método que graba en un fichero la comanda realizada poniendo cual es el número de la comanda y tanto los artículos pedidos como las observaciones.
- **Boolean comprobarFinJuego():** método que comprueba si el juego ha terminado mirando si el número de intentos es 0. Si es 0 devuelve true y sino devuelve false.

Clase Juego:

En esta clase tenemos todos los métodos relacionados con el juego. Como atributos tenemos el número de intentos para conseguir pareja, una variable que indica si ya se ha jugado al juego o no, un objeto tablero, el número de casillas seleccionadas en cada momento, dos variables de tipo casilla; una con la primera casilla seleccionada para hacer pareja en ese intento y otra con la segunda, una lista con los premios obtenidos y un objeto de tipo carta. Además, tenemos una constante con el tamaño del tablero que en este caso es 16.

Como métodos más importantes tenemos:

- **Void iniciarJuego():** método que inicializa los valores del juego para una nueva reserva, poniendo el número de intentos a 5, que no se ha jugado, las dos casillas seleccionadas a null y el número de casillas seleccionadas a 0.
- **Void seleccionCasilla(int indice):** método que suma uno a las casillas seleccionadas, en caso de que las casillas seleccionadas sean 1, almacena la casilla del índice pasado como parámetro como la primera seleccionada, y si el número de casillas seleccionadas es 2, pone esa variable a 0 para empezar una nueva pareja y almacena la casilla del índice pasado como parámetro como la segunda casilla seleccionada.

- **Boolean HayPareja():** método que comprueba si hay pareja viendo si el tipo de las dos casillas seleccionadas es el mismo. Devuelve true si la hay y false en cualquier otro caso.
- **Void añadirPremio():** en caso de que haya pareja se añade un premio que dependerá del número de intentos restantes. Dependiendo el número de intento se añadirá un tipo de artículo u otro. Para ello se llamará a otros métodos que dependiendo de la categoría crean una lista con los elementos de esa categoría y escogen uno aleatorio de esa lista.

Clase Tablero:

Clase que crea un tablero para el juego y que tiene como atributo el tamaño, que en nuestro caso siempre será 16, y un array con todas las casillas del tablero.

Como métodos más importantes tenemos:

- **Void colocarParejas():** método que coloca dos casillas de cada uno de los 8 tipos que hay, y que corresponden con las 8 posibles parejas. Para ello en un bucle que itera 2 veces habrá un método por cada tipo de casilla el cual añadirá en una posición aleatoria del tablero, que este vacía, una casilla de ese tipo. Si la casilla aleatoria no está vacía se busca otra.

Clase Casilla:

Clase que crea cada una de las casillas del tablero y que declara un tipo enumerado con los 8 tipos de parejas y el tipo vacío que será el asignado a la casilla mientras no se la haya asignado ninguno de los otros tipos. Como atributos tiene el tipo de casilla que será uno de los del tipo enumerado antes citado, un string con la dirección de la imagen, la cual dependerá del tipo de la casilla, y una variable que indica si se ya ha sido encontrada como pareja.

Como métodos más importantes tenemos:

En esta clase solo tenemos los getters y setters correspondientes a cada uno de los atributos.

Clase Carta:

Clase que crea la carta de la pizzería. Declara como atributos la lista de los artículos de la carta, y la lista de los artículos filtrados que son los que corresponden a las categorías marcadas en el filtro. Además, también se declara como constante la url del fichero que contiene la carta.

Como métodos más importantes tenemos:

- **Void cargarArtículos():** mete en la lista de artículos los diferentes artículos sacados del fichero carta.dat.
- **Void añadirArticulosFiltrados(String actionCommand) y void eliminarArticulosFiltrados (String actionCommand):** métodos que añaden o eliminan de la lista de artículos filtrados los artículos cuya categoría se corresponda con la pasada por parámetro, ya que el actionCommand de cada artículo es su categoría.
- **Boolean estaEnFiltro(int indice):** método que comprueba si el elemento del índice pasado como parámetro esta en la lista de artículos filtrados. Si lo está devuelve true y si no lo está devuelve false.
- **Void limpiarListaArticulosFiltrados():** vacía la lista de artículos filtrados para reiniciar el filtro sin que ninguna categoría este seleccionada.
- **Void añadirArticulosFiltroSoloIntolerancias(String actionCommand):** método que añade a la lista de artículos filtrados los artículos de la categoría que se corresponda con el actionCommand y los cuales también sean posible adaptar a intolerancias.

Clase Artículo:

Clase que crea cada uno de los artículos de la carta, especificando en sus atributos el código del artículo, su denominación, su descripción explicando todo lo que lleva, si es posible adaptarlo o no a intolerancias, su categoría (pizza, postre…), su precio y el número de unidades pedidas de ese artículo.

Como métodos más importantes tenemos:

En esta clase solo tenemos los getters y setters correspondientes a cada uno de los atributos.

Clase Reservas:

Clase que crea la lista de las reservas del restaurante, teniendo como atributo esa lista de reservas. Además, también tiene una constante con la url del archivo que contiene la lista de reservas.

Como métodos más importantes tenemos:

- **Void cargarReservas():** mete en la lista de reservas todas las reservas que hay en el fichero reservas.dat.
- **Boolean confirmarReserva(String dni, String numReserva):** método que comprueba si la reserva correspondiente al dni y número de reserva pasados

como parámetros se corresponde con alguna reserva registrada, devuelve true en caso de que si y false en caso de que no.

- **Void eliminarReserva(String dni, String numReserva):** elimina de la lista de reservas la reserva correspondiente al dni y número de reserva pasados como parámetros. También escribe un nuevo fichero reservas.dat eliminando esa reserva.

Clase Reserva:

Clase que crea cada una de las reservas con sus datos, el dni asociado a la reserva y el número de reserva.

Como métodos más importantes tenemos:

En esta clase solo tenemos los getters y setters correspondientes a cada uno de los atributos.

Clase ClavesEmpleados:

Clase que contiene cada una de las claves que corresponden a cada empleado y que les permite preparar todo para poder atender una nueva reserva. Tiene como atributo la lista de claves. También declara como contante la url del fichero que contiene las claves.

Como métodos más importantes tenemos:

- **Void cargarClaves():** carga en la lista de claves las diferentes claves de todos los empleados.
- **Boolean confirmarClave(String nombre, String contraseña):** comprueba que el nombre y la contraseña pasados como parámetros se corresponden con algunas de las claves de la lista.

Clase ClaveEmpleado:

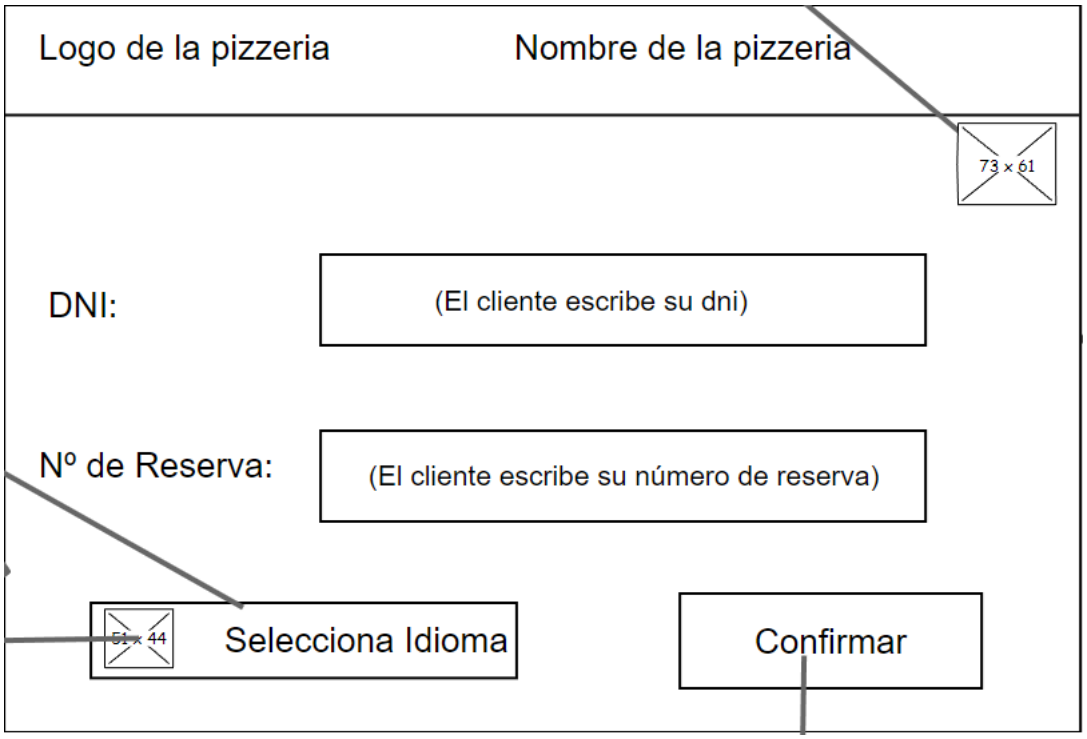
Clase que crea la clave de un empleado que contendrá su nombre y la contraseña que se le asigna a cada empleado.

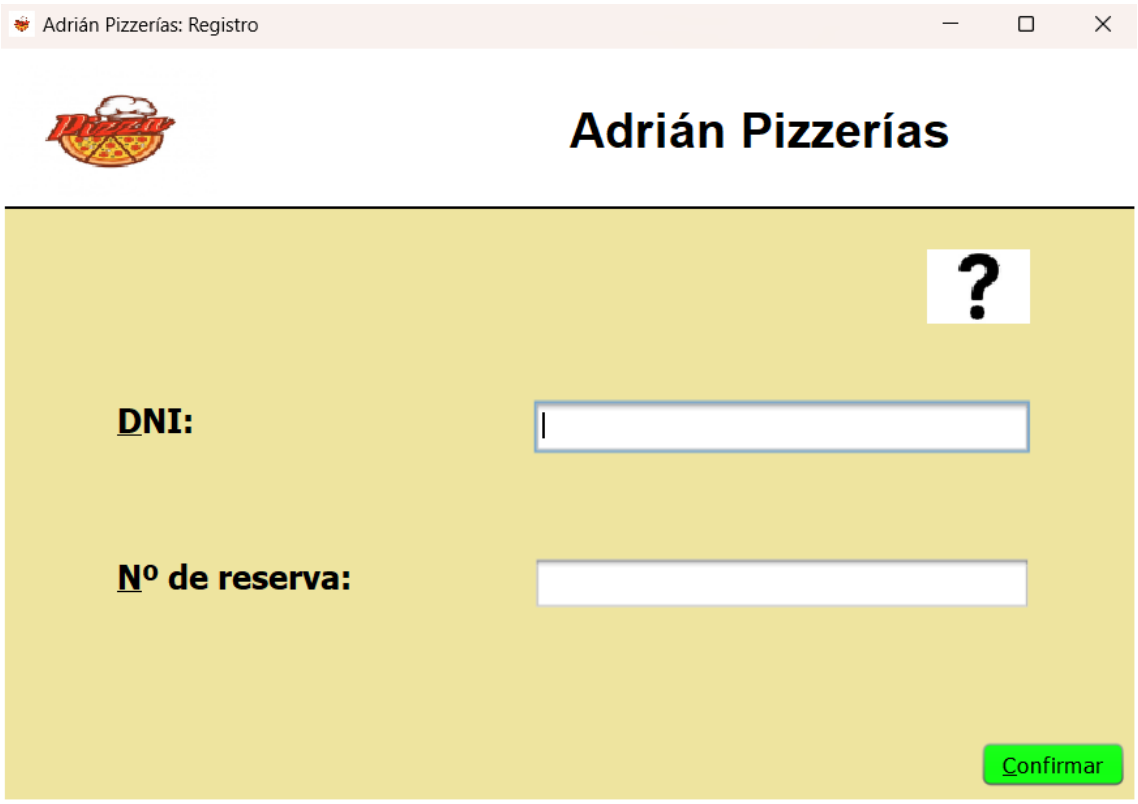
Como métodos más importantes tenemos:

En esta clase solo tenemos los getters y setters correspondientes a cada uno de los atributos.

INTERFAZ:

Ventana registro:





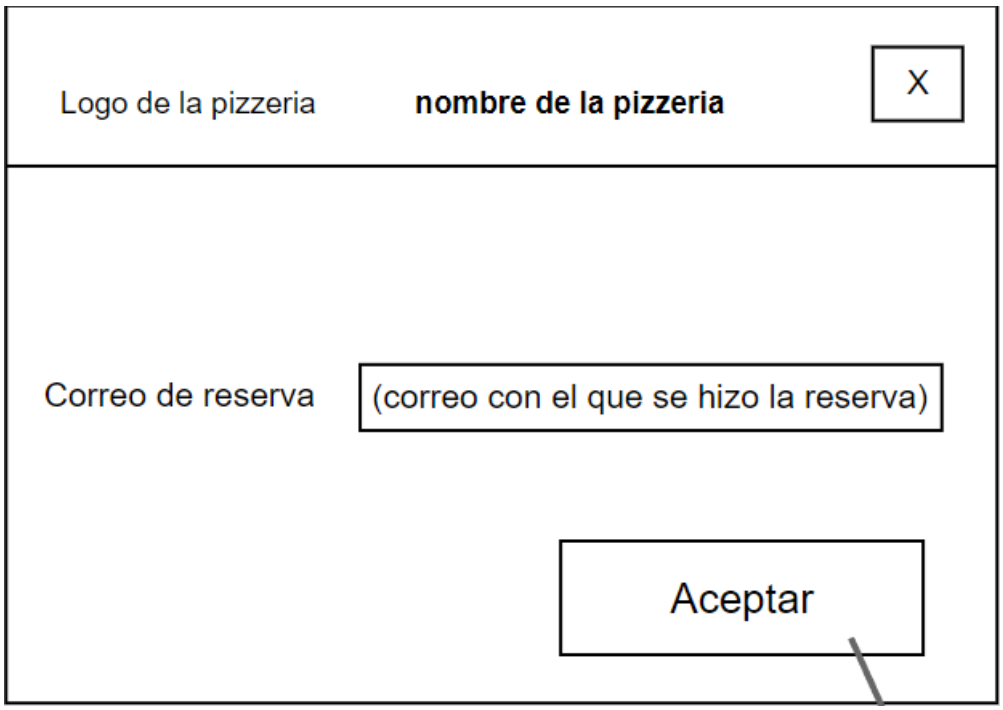
De esta ventana podemos destacar las labels y los textFields que tienen asociados para escribir el dni y el número de reserva, siendo estas las mejores opciones a la hora de tener que introducir datos(que se escriben en el textField) sobre unos

campos(escritos en las labels). Hay también un botón con signo de interrogación que era la mejor opción para poder abrir una nueva ventana con la intencionalidad de recuperar el número de reserva.

Cambios respecto al wireframe:

En este caso el botón que te llevaba a la ventana para escoger el idioma no está ya que decidí no hacer la opción de internacionalizar.

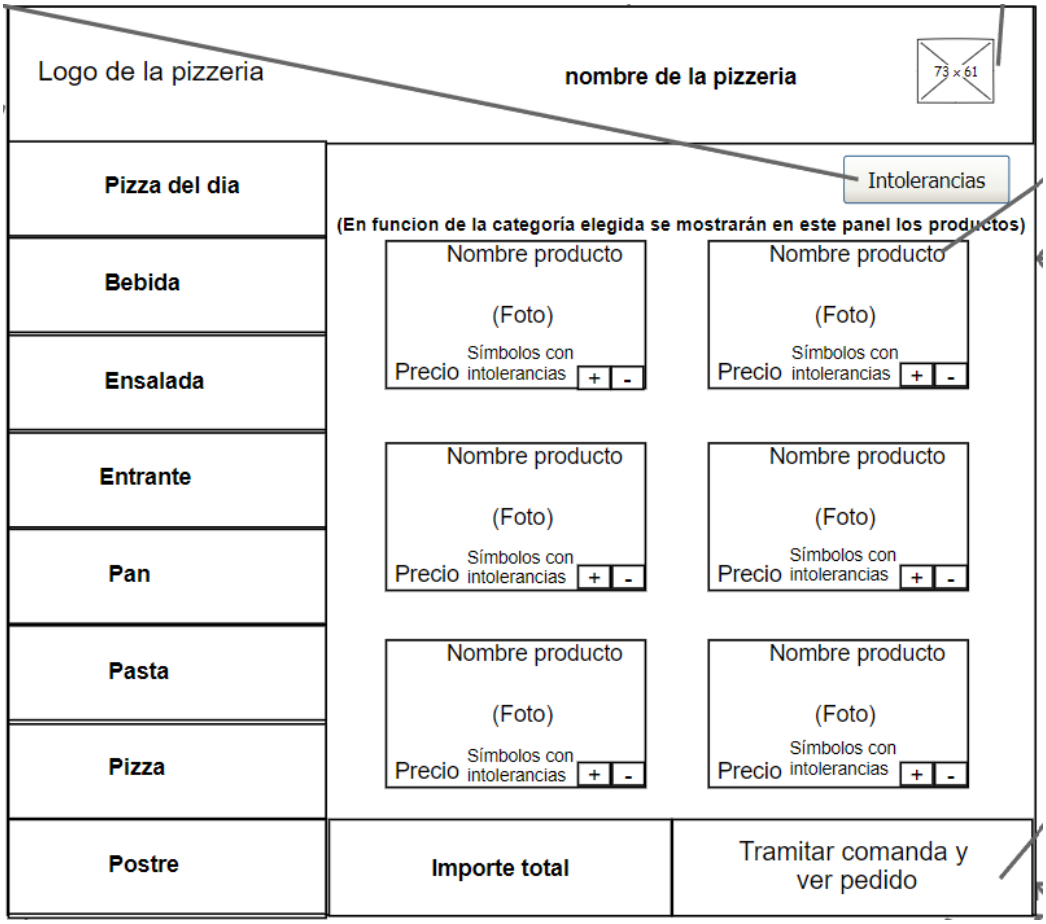
Ventana recuperar datos de reserva:

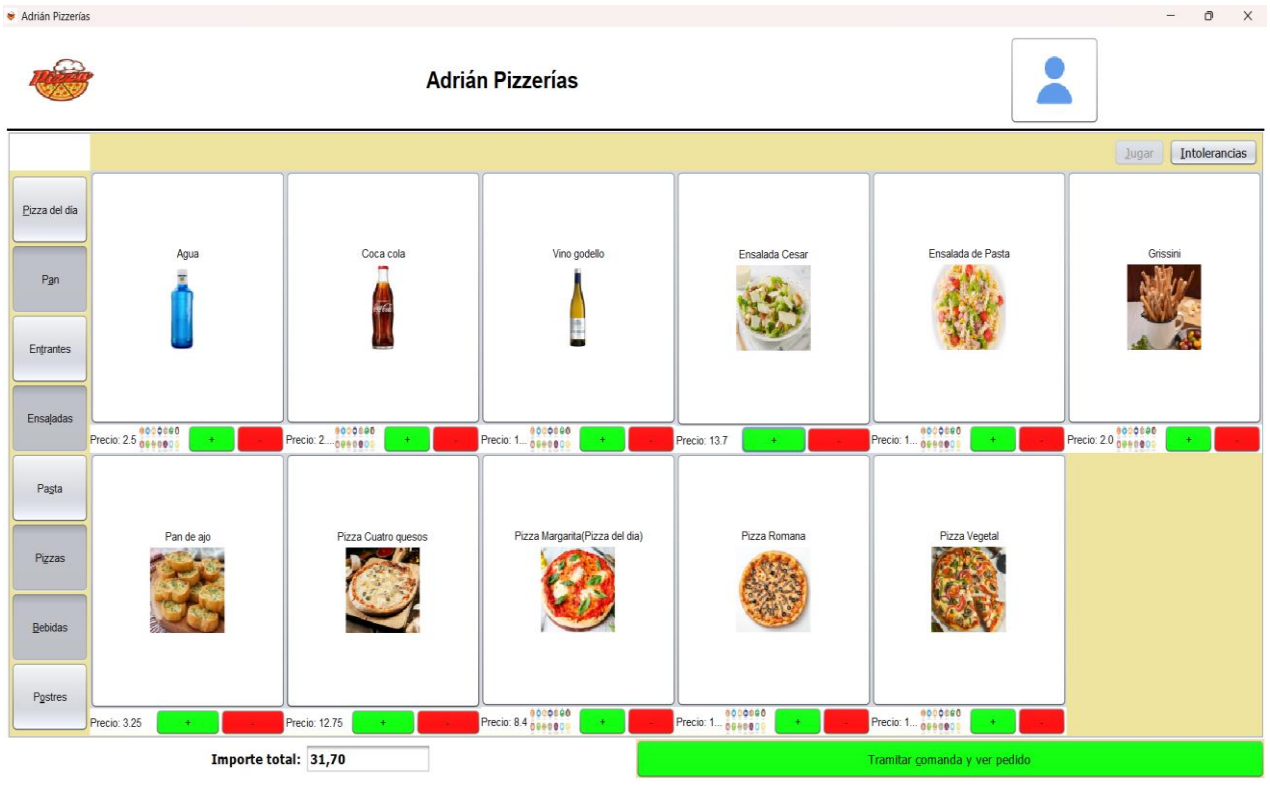




En esta ventana destacamos a igual que en la ventana anterior la label y el textField ya que es la mejor opción para la introducción de datos.

Ventana menú principal:





En este caso tenemos varios elementos importantes en la ventana:

- Los artículos que se muestran los he creado dinámicamente mediante paneles, y dentro de uno de los paneles se crea dinámicamente también la etiqueta con el precio, las intolerancias y los botones de añadir y eliminar. Esta es la mejor forma ya que en caso de que se añadieran o quitaran artículos de la carta no habría que hacer ningún cambio.
- Para los botones del filtro he optado por toggleButtons ya que al tener que poderse marcar varias categorías a la vez era la forma más cómoda, así como también hice con el botón para el filtro de las intolerancias. Así también se puede ver cuando el filtro o cuando las categorías están marcadas o no.
- También tenemos un sitio para ver el precio total y un botón para tramitar el pedido y poder ver el resumen de este.

Cambios introducidos respecto al wireframe:

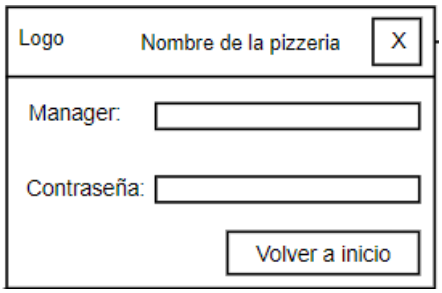
- El orden de los botones del filtro lo he cambiado simplemente porque así el orden es más lógico, empezando por el pan y acabando por los postres.
- En cuanto a la pizza del día, en el wireframe no había puesto ninguna forma para mostrarla así que he tenido que diseñar un nuevo panel para mostrar cual es la pizza del día, además que en el filtro de pizzas se muestra encima de su imagen cual es.



- Añadí un botón de jugar, ya que si los clientes decidían no jugar después de la primera comanda, si querían jugar más tarde estaban obligados a hacer otra comanda. Por esta razón he añadido el botón de jugar al lado del filtro de intolerancias que solo estará disponible si el cliente ha hecho una comanda y todavía no ha jugado al juego.
- El botón de intolerancias ahora es un toggleButton en vez de abrir otra ventana, ya que como no hay que especificar las intolerancias, es solo en general decir si se puede adaptar a intolerancias, el filtro que yo había puesto

en el wireframe no se puede hacer, por lo que opte por utilizar lo mismo que con el filtro normal ya que así se ve claramente cuando esta puesto el filtro y cuando no.

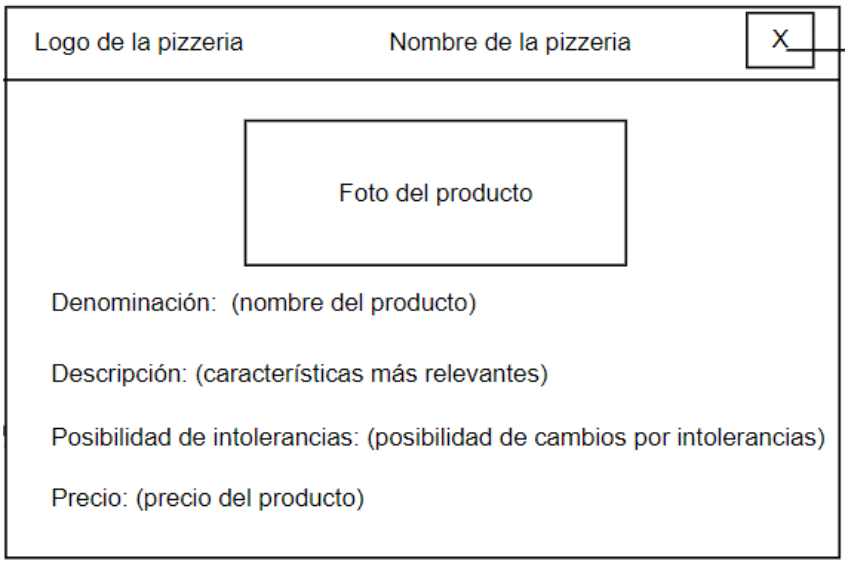
Ventana nueva reserva:

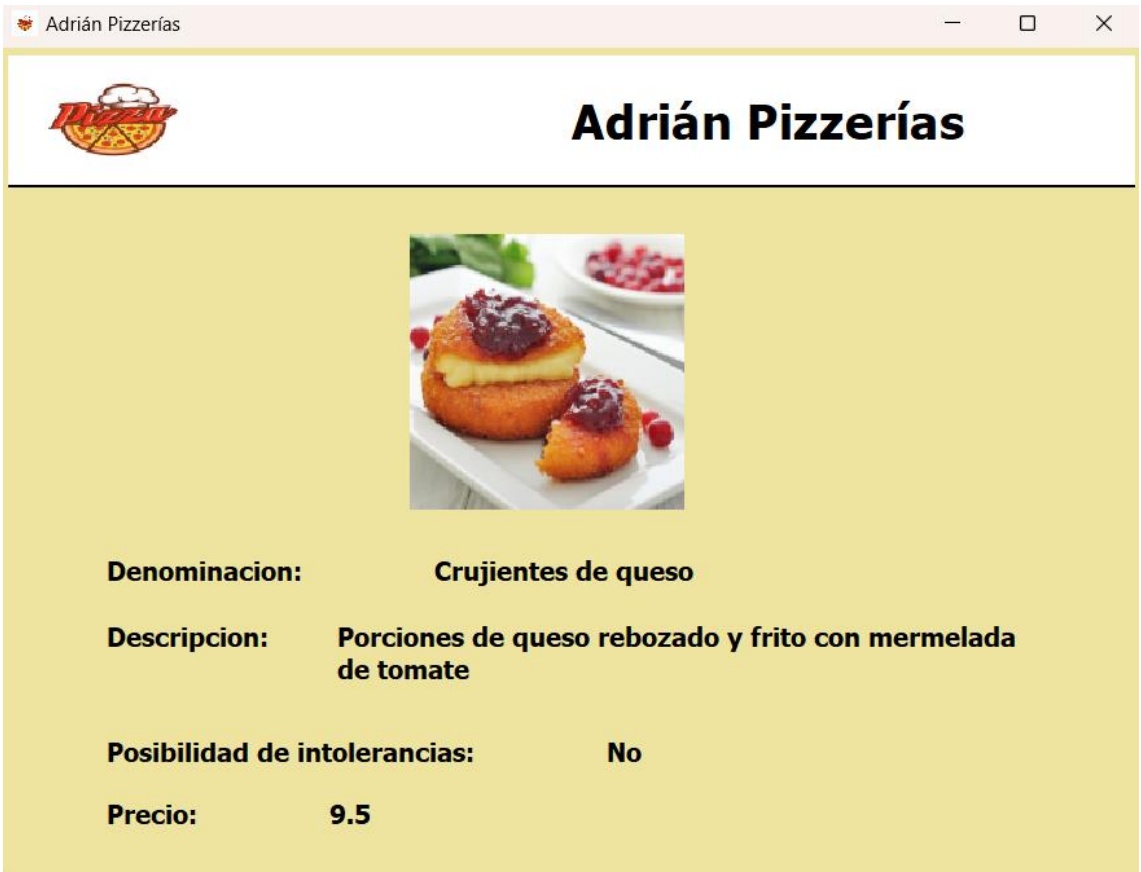




Como elementos más importantes volvemos a tener las labels y los textFields ya que nuevamente solo hay que introducir datos.

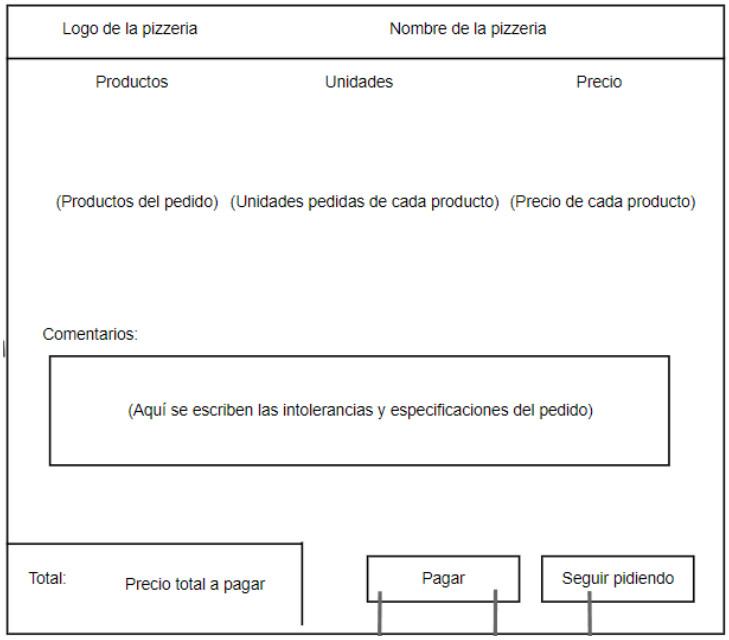
Ventana información artículos:

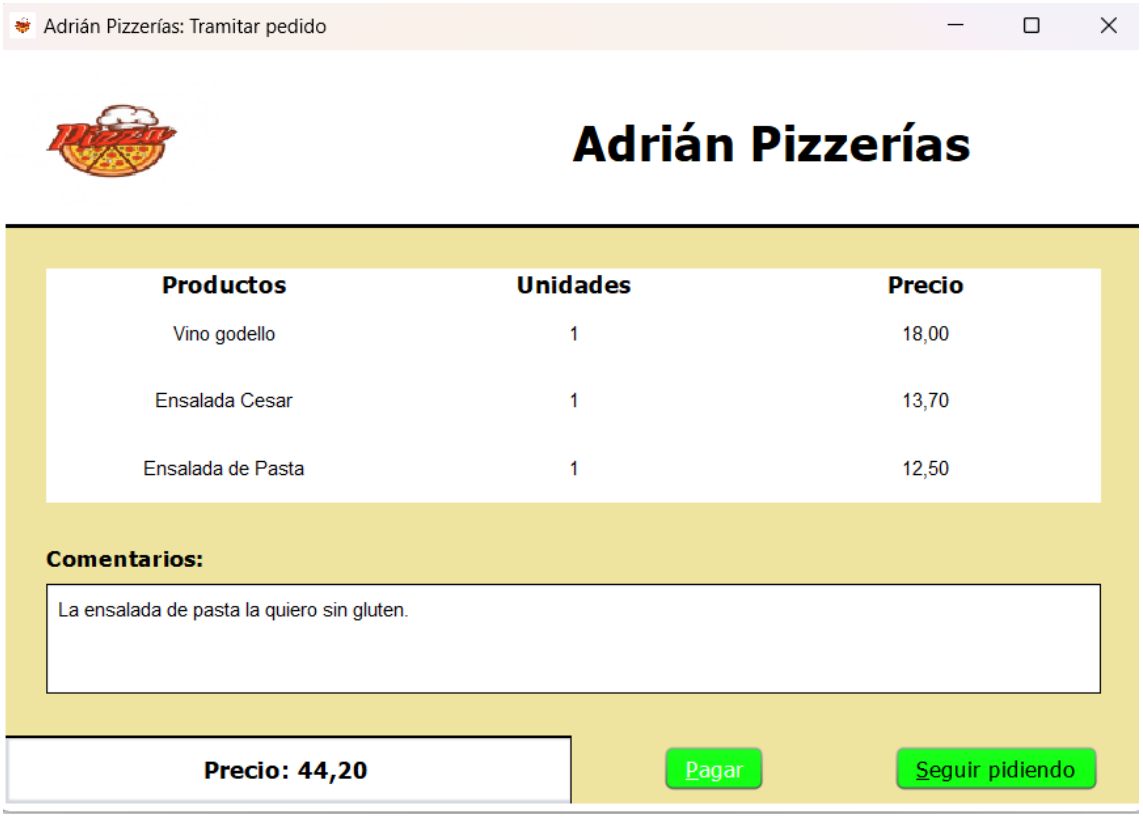




En este caso tanto para la foto y para la información del artículo he utilizado labels ya que es información que no se puede modificar en tiempo de ejecución. Únicamente he utilizado un textArea para el texto de la descripción ya que el número de ingredientes puede variar y ocupar diferente espacio en un artículo que en otro necesitando más lineas.

Ventana tramitar pedido:





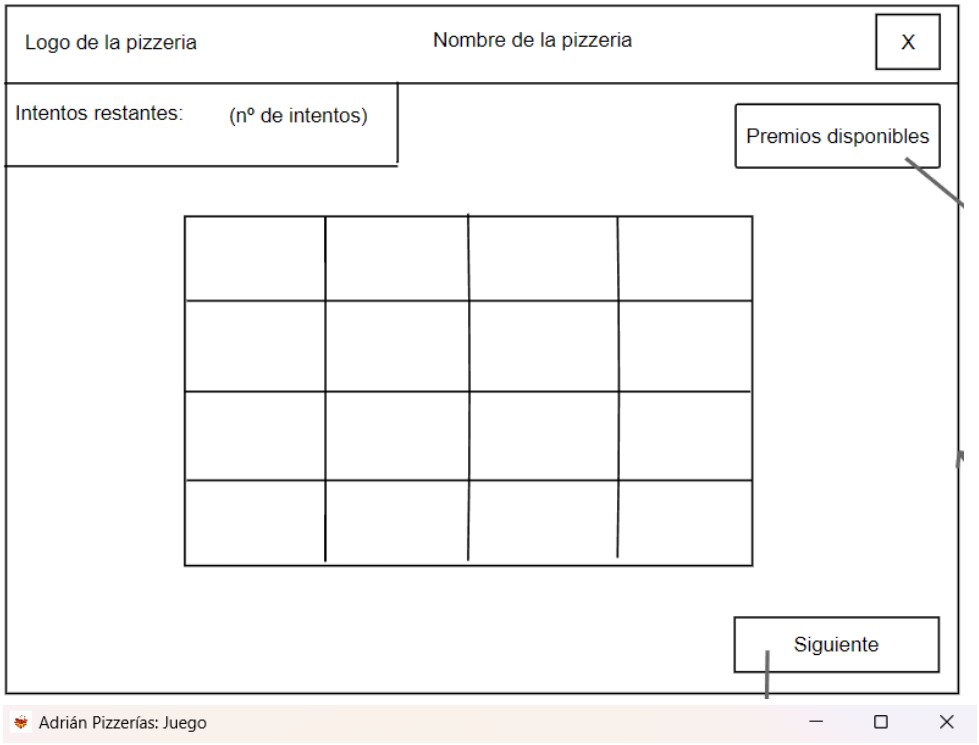
En este caso tenemos varios elementos importantes:

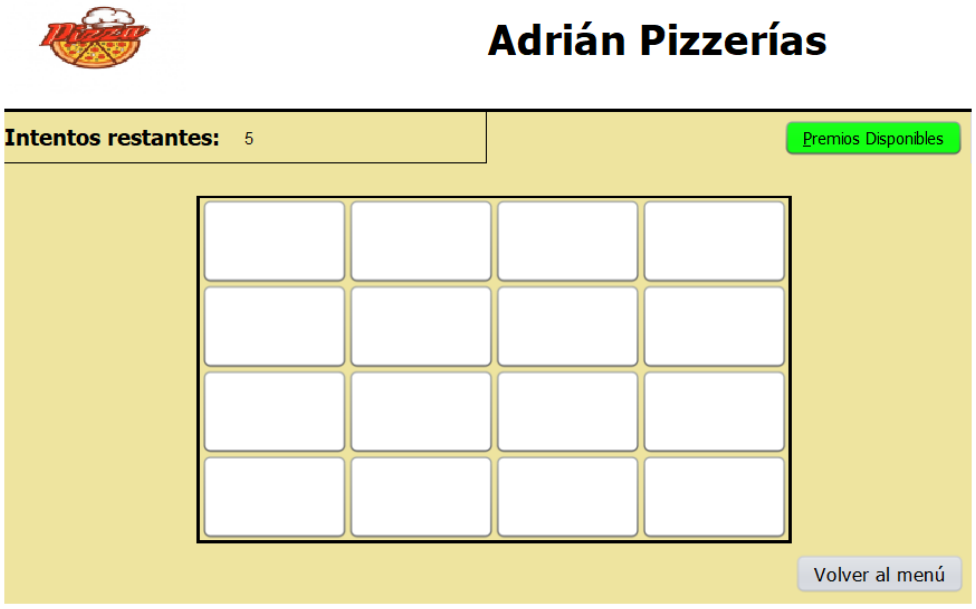
- La zona donde está el resumen del pedido es un scrollPane al que se le van añadiendo tres paneles dinámicamente, uno para cada apartado, dependiendo de los artículos diferentes que hay en el pedido. Lo he escogido así y no por ejemplo con un textArea y escribiendo ahí el resumen ya que sino

no me quedaban los apartados alineados por las diferentes longitudes en las denominaciones de los productos.

- Para los comentarios he escogido un textArea y no un textField ya que la longitud de los comentarios puede ser muy grande y ocupar más de una linea.
- Después tenemos un sitio para mostrar el precio total, un botón para pagar y otro por si el cliente quiere añadir algo más al pedido.

Ventana juego:





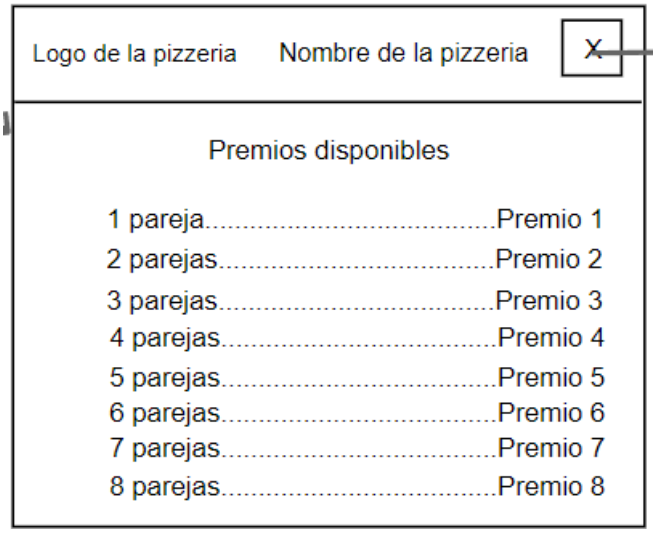
Como elementos más importantes tenemos:

- El tablero que está formado por botones, que es la mejor opción para este juego ya que hay que interactuar con ellos por lo que una label no es una buena opción. Estos botones se crean dinámicamente ya que el tamaño del tablero puedo cambiar.
- También tenemos un txField arriba donde se muestran los intentos restantes, un botón para mostrar los premios disponibles, y un botón volver al menú que solo estará activo cuando se acabe el juego.

Cambios introducidos respecto al wireframe:

- Cambié el texto del botón de abajo a la derecha de “Siguiente” a “Volver al menú”, ya que así se veía mucho más claro cual era la funcionalidad de ese botón, sino no quedaba claro que iba a pasar al pulsarlo.

Ventana premios disponibles:





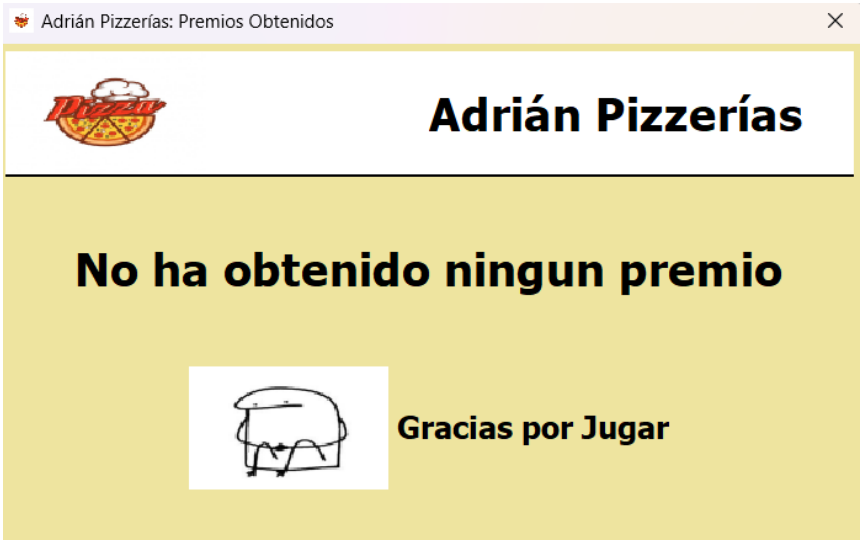
En esta ventana simplemente he utilizado labels para mostrar la información de los premios que se pueden conseguir, ya que es texto que no se puede modificar en tiempo de ejecución y meramente informativo.

Cambios respecto al wireframe:

- Como el modo de ganar premios es diferente al que yo pensaba, en el wireframe ponía los premios por parejas y ahora los pongo por intentos ya que es como lo especifica en los requisitos.

Ventana premios obtenidos:

En el caso de esta ventana, no estaba contemplada en el wireframe ya que no se sabía que un requisito iba a tener que ser mostrar los premios disponibles, por lo que he creado una ventana que se abra al terminar el juego y muestre de forma dinámica los premios obtenidos, ya que no se sabe cuántos premios se van a obtener, o que no se ha obtenido ningún premio.





PRUEBAS:

Persona (Nombre en la ficha persona-escenario): Laura

Dificultades encontradas: Ninguna dificultad encontrada

Acciones de mejora en la interfaz: Ninguna

Persona: Antonio

Dificultades encontradas: Ninguna dificultad encontrada

Acciones de mejora en la interfaz: Ninguna

Persona: Nathalie

Dificultades encontradas: Ninguna dificultad encontrada

Acciones de mejora en la interfaz: Ninguna

Persona: María

Dificultades encontradas: Ninguna dificultad encontrada

Acciones de mejora en la interfaz: Ninguna

Persona: Saray

Dificultades encontradas: Ninguna dificultad encontrada

Acciones de mejora en la interfaz: Ninguna

Persona: Jose Ramón

Dificultades encontradas: Ninguna dificultad encontrada

Acciones de mejora en la interfaz: Ninguna

Persona: Antonio

Dificultades encontradas: Ninguna dificultad encontrada

Acciones de mejora en la interfaz: Ninguna

Persona: Carlota

Dificultades encontradas: Ninguna dificultad encontrada

Acciones de mejora en la interfaz: Ninguna